A Gentle Introduction to Algorithms & Data Structures

Tim Book

GENERAL ASSEMBLY

Prerequisites

While this is a standalone lesson that is mostly separate from other DSB course content, the following two skills are required:

- Strong base Python ability
- Understanding of OOP (ie, the “class” keyword)

 © 2024 General Assembly

Agenda

- Why isn’t this topic part of our core material?
- What do we mean by “algorithm”?
- “Big-O” notation
- What do we mean by “data structure”?
- Example Case: Searching

 © 2024 General Assembly

What is this topic?

The topic of algorithms and data structures is a very important topic typically taught to **freshmen computer science majors**. It's a really tough class that's often used as a weed-out for the major.

While some of the use-cases taught as part of the topic are superfluous (they'll either never happen in practice or are already handled under-the-hood for us), it's a fascinating topic that can show up in unexpected places in practice.

In other words: It'll make you a better programmer!

 © 2024 General Assembly

Why is it being relegated to a CYOA lesson, then?

The short answer is: We don't need it! (usually)

As data scientists, most of our code is declarative rather than algorithmic. That is, we're often saying things like:

"Do this, then do that" (Ex: "Do a train-test split, then gridsearch")

rather than:

"Loop over this array, retain certain values, call this recursive function, ..."

 © 2024 General Assembly

Why is it being relegated to a CYOA lesson, then?

That is, most **algorithms** are being done behind the scenes as part of functions that are pre-built for us. We're rarely writing them ourselves.

However, that's not always true! Sometimes we have to write twisty code that can benefit from speedups if we know what we're doing.

Maybe after this lesson, you'll see how to spot such occasions.

 © 2024 General Assembly

Ok, so what are...

Algorithms

Algorithms

Simply put, an **algorithm** is a pattern our routine we follow to solve a task. You probably already know a few:

- Gradient descent
- Making a peanut butter and jelly sandwich
- *k*-Fold Cross Validation
- *k*-Means Clustering

Algorithms

In general computer science, people typically have a more specific definition of an algorithm. They're generally referring to a set of instructions that is guaranteed to solve a specific type of problem. Classical problems in CS are:

- Sorting a set of numbers
- Searching through a set of numbers for a value
- Shortest distance between two points (graph traversal)
- Game playing (Ex: sudoku solvers, tic tac toe players)

Big O Notation

Algorithm Speed

First, we need a way to measure how “fast” an algorithm is - or how could we say one is better than another?

Sometimes considered a scary topic, let’s discuss **Big O notation**.

 © 2024 General Assembly

Landau's O Notation

Big O notation, sometimes Landau's O notation, is a way of categorizing the rate of increase of a function as $n \to \infty$. Formally:

A function f is $O(g(n))$ if:

$$f(n) \leq c \cdot g(n) \text{ for some } c \text{ as } n \to \infty$$

 © 2024 General Assembly

Examples

Function	Big O Categorization
$f(n) = n^2$	$O(n^2)$
$f(n) = 500n^2 - 324n + 123$	$O(n^2)$
$f(n) = \frac{1}{100}n^5 + n^3 - 700$	$O(n^5)$
$f(n) = n + n\log n + \log n$	$O(n\log n)$
$f(n) = \log n + \frac{1}{n}$	$O(\log n)$
$f(n) = 2^n + 999999n^5$	$O(2^n)$

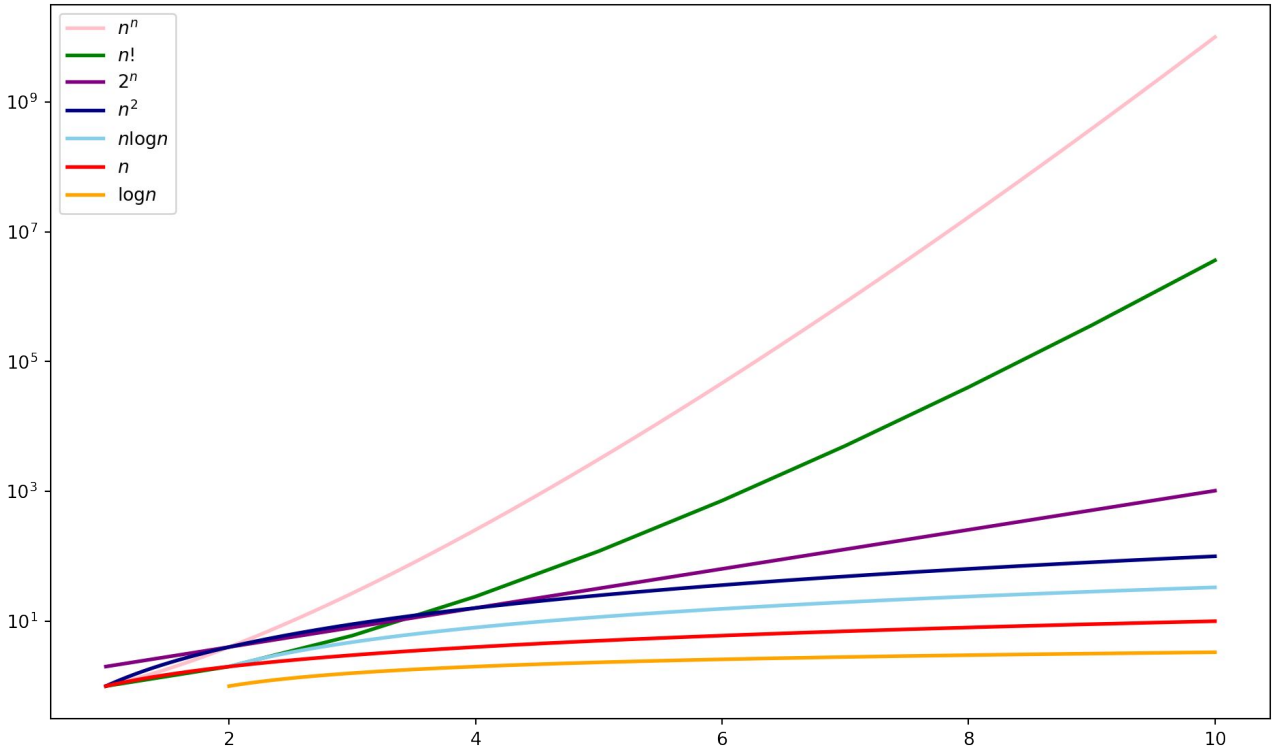
Note: All logs today are base 2 by default. Not e!

Additional Big O Notes

- Big O categorization is an “upper bound” or “worst case scenario” for a function over the long run. This means that: $f(n) = 3n^2$ is $O(n^2)$, but it’s also $O(n^3)$!
- The sum of two functions have the Big O categorization of the “greater” function. Ex: $O(n^4) + O(n^3) = O(n^4)$
- “Constant time” functions are represented by $O(1)$ - these are functions that have a deterministic finish time
- In some fields, functions that go to 0 are of interest, so you’ll see things like $O(1/n)$

 © 2024 General Assembly

Big O Hierarchy



Faster
$O(n^n)$
$O(n!)$
$O(2^n)$
$O(n^2)$
$O(n \log n)$
$O(n)$
$O(\log n)$
Slower

Realistic Example

Consider a list of integers of length *n*. Our task is to sort this list from smallest to largest. How “fast” is this algorithm?

```
sorted_list = []
previous_min = -1

for i in range(len(my_list)):
    current_min = 999
    for item in my_list:
        if item < current_min and item > previous_min:
            current_min = item
    sorted_list.append(current_min)
    previous_min = current_min
```

Quicknotes: This algorithm loops through our list, each time finding the next smallest number, and one-by-one appends those minima onto our new list, thus creating a sorted result.

Realistic Example

Let's consider the **worst case scenario**:

```
sorted_list = []
previous_min = -1

for i in range(len(my_list)):
    current_min = 999
    for item in my_list:
        if item < current_min and item > previous_min:
            current_min = item
    sorted_list.append(current_min)
    previous_min = current_min
```

Realistic Example

Let's consider the **worst case scenario**:

```
sorted_list = []
previous_min = -1

for i in range(len(my_list)):
    current_min = 999
    for item in my_list:
        if item < current_min and item > previous_min:
            current_min = item
    sorted_list.append(current_min)
    previous_min = current_min
```

Looping through *n* items

 © 2024 General Assembly

Realistic Example

Let's consider the **worst case scenario**:

```
sorted_list = []
previous_min = -1

for i in range(len(my_list)):
    current_min = 999
    for item in my_list:
        if item < current_min and item > previous_min:
            current_min = item
    sorted_list.append(current_min)
    previous_min = current_min
```

Looping through *n* items

Looping through *n* items within each outer loop

Realistic Example

Let's consider the **worst case scenario**:

```
sorted_list = []
previous_min = -1

for i in range(len(my_list)):
    current_min = 999
    for item in my_list:
        if item < current_min and item > previous_min:
            current_min = item
    sorted_list.append(current_min)
    previous_min = current_min
```

Looping through *n* items

Looping through *n* items within each outer loop

This is n^2 total "calculations". We're in $O(n^2)$ time!

Looping Rule of Thumb

Sometimes it's hard to look at an algorithm and determine its big O "speed". A general rule of thumb is that **nested loops are bad and slow!**

Two loops, one nested inside the other? $O(n^2)$!

Three loops, all nested? $O(n^3)$!

 © 2024 General Assembly

Big O Classes for common math problems

Operation	Big O Categorization
Simple math operation (Ex: addition)	$O(1)$
Taking a square root	$O(\log n)$
Dot product between two *n*-dim vectors	$O(n)$
Multiplying two *n* x *n* matrices	$O(n^3)$ (later optimized to) $O(n^{2.3728596})$
Inverting an *n* x *n* matrix	$O(n^3)$ (later optimized to) $O(n^{2.373})$

 © 2024 General Assembly

Data Structures

Data Structures

A **data structure** is a way of organizing and storing data in a computer's memory so that it can be used efficiently for certain algorithms.

For example, the DataFrame data structure provided by Pandas allows us to "efficiently" (from our perspective) manipulate data.

How about a more "computer science-y" example: How can we store a simple list of numbers?

 © 2024 General Assembly

Arrays

One common way of storing data is in an **array**.

7	-3	5	11	0	1

Arrays

One common way of storing data is in an **array**. Arrays store data sequentially in memory. The language allocates this memory in RAM for you.

memory “slot”	6001	6002	6003	6004	6005	6006
array value	**7**	**-3**	**5**	**11**	**0**	**1**

Arrays

Because of this setup, it's really easy to...

- **index** an array. You simply call **arr[3]** and your computer can deduce the location of the 4th element (6001 + 3 = 6004)
- **edit** an array value. It's easy for your computer to edit the value stored in a certain memory slot.

memory "slot"	6001	6002	6003	6004	6005	6006
array value	**7**	**-3**	**5**	**11**	**0**	**1**

 © 2024 General Assembly

Arrays

However, it's really difficult to **insert** a new value into the array. Suppose you want to add a value between the 5 and the 11. How would you do this?



memory "slot"	6001	6002	6003	6004	6005	6006
array value	**7**	**-3**	**5**	**11**	**0**	**1**

Arrays

Do insert into an array, you'd need to duplicate the array somewhere else in memory with an extra space, assign to that extra space, and then de-allocate the old allocation. This is incredibly inefficient and wasteful!

memory "slot"	6001	6002	6003	6004	6005	6006
array value	**7**	**-3**	**5**	**11**	**0**	**1**

memory "slot"	8001	8002	8003	8004	8005	8006	8007
array value	**7**	**-3**	**5**	**3**	**11**	**0**	**1**

 © 2024 General Assembly

Linked Lists

If you're going to be appending and inserting with your data, it will be more efficient to instead use a **linked list**. Instead of storing data sequentially in RAM, each piece of data contains two values: the value itself, and the location of the *next* piece of data.



 © 2024 General Assembly

Linked Lists

Now,

- **indexing** is slightly more difficult (you can't just mathematically deduce where an element is)
- **editing** is still easy

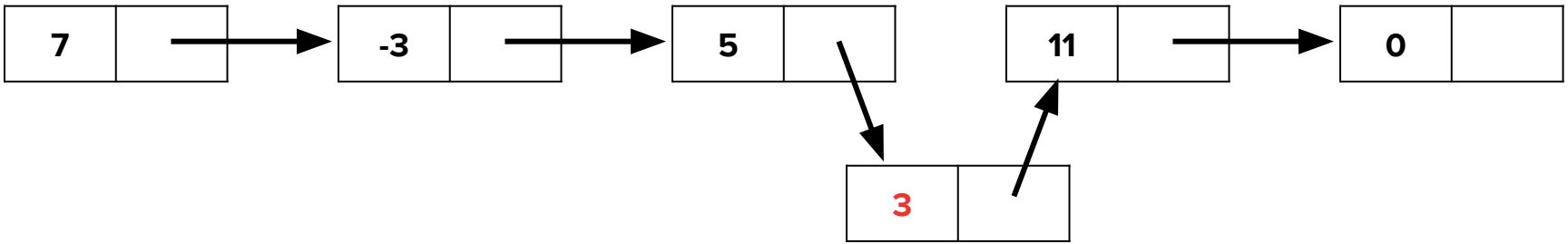


 © 2024 General Assembly

Linked Lists

But...

- **inserting** is now easy and efficient! You can change the pointer to a new element, and have the new element point to its new “next” element.



Linked Lists vs Arrays

Both data structures hold the same information: a sequence of integer values, but in very different ways. Neither is “better”, but one is a better choice depending on what you’re going to do with it.

This is why talking about algorithms often involves talking about data structures! Some algorithms become possible only with a fancy data structure!

 © 2024 General Assembly

Linked Lists vs Arrays (using the representations discussed so far)

Task	Array	Linked List
indexing	$O(1)$	$O(n)$
insertion/deletion	$O(n)$	$O(1)$
search	$O(n)$	$O(n)$

Python Note

Maybe you're wondering... what's a Python list then?

Python lists are neither! They're actually **dynamically allocated arrays** - an attempt to get the best of both worlds (to a degree).

If you want to do linked list-like operations super efficiently, you can use a double-ended queue, found in **collections.deque()**.

Case Study: Binary Search Trees



Searching

Consider the task of searching through an (unsorted) array to determine if a certain value is inside it. Here's a simple algorithm:

```
def linear_search(arr, target):
    for value in arr:
        if value == target:
            return True
    return False
```

Since there's only one loop, we'll call this **linear search** since it will be ***O(n)***. In the worst case scenario, you'll have to loop through the whole array (*n* elements).

 © 2024 General Assembly

Can we beat *O*(*n*)?

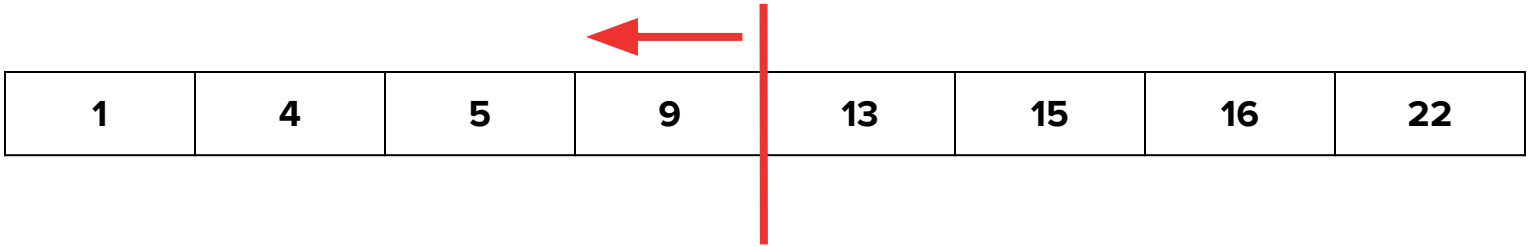
What if our array was sorted? Let's say we're trying to find the 5 in this sorted array:

1	4	5	9	13	15	16	22

 © 2024 General Assembly

Can we beat *O*(*n*)?

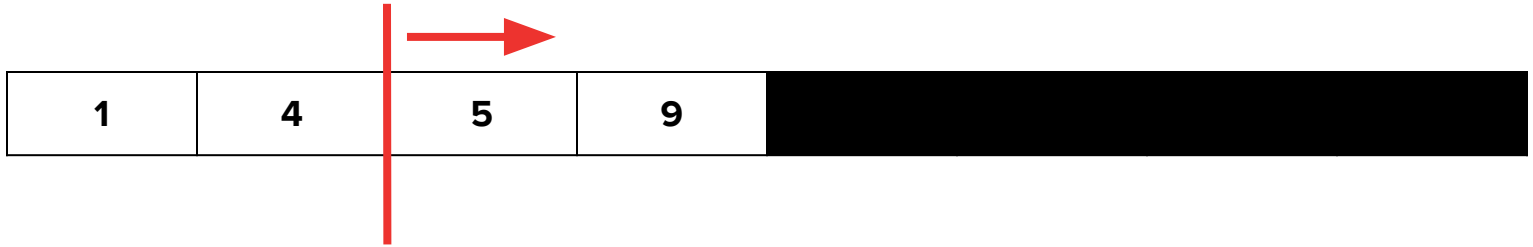
Since the array is sorted, we can just cut the array in half and see if we should go left or right. Since the cut happens between 9 and 13, we should look left.



 © 2024 General Assembly

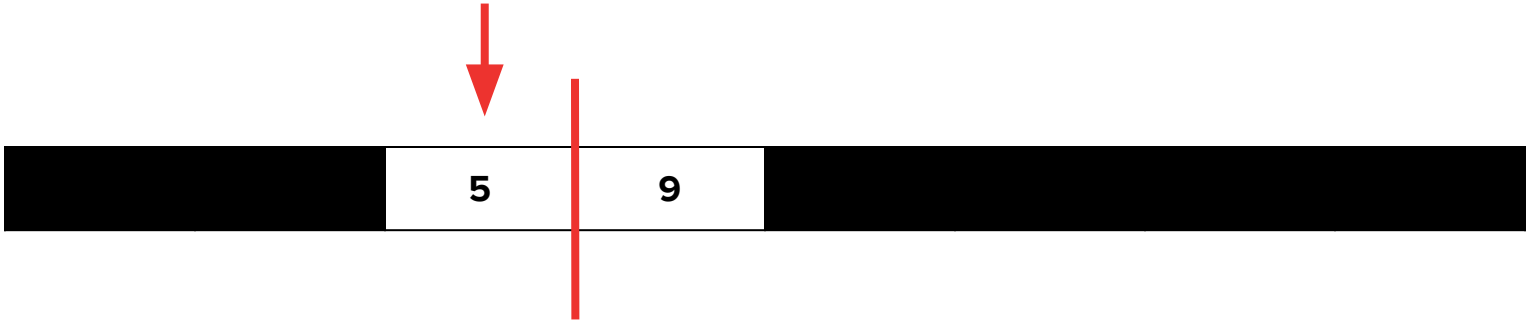
Can we beat *O*(*n*)?

We do the same with the remaining array, halving it each time until we find 5 (or determine it can't exist).



Can we beat *O*(*n*)?

We do the same with the remaining array, halving it each time until we find 5 (or determine it can’t exist).



 © 2024 General Assembly

Can we beat *O*(*n*)?

In the special case where the array is sorted, we can beat *O*(*n*)! Since we're halving the array each time, it's *O*(log *n*) which is considered very fast!

But... of course usually our data aren't sorted. And sorting can also be a very expensive operation (usually slower than *O*(*n*)).

The trick is to develop a data structure that will allow us to "half" our data at each step without sorting it! We'll create a **binary tree**!

 © 2024 General Assembly

Binary Tree

Take the unsorted array and begin by placing its first element in the “root” node of a binary tree.

13	9	16	22	15	1	5	4



 © 2024 General Assembly

Binary Tree

For each successive element, descend the tree, putting lower numbers to the left, larger numbers to the right.

13	9	16	22	15	1	5	4

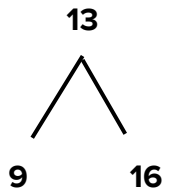


 © 2024 General Assembly

Binary Tree

For each successive element, descend the tree, putting lower numbers to the left, larger numbers to the right.

13	9	16	22	15	1	5	4

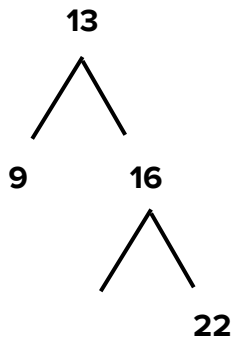


 © 2024 General Assembly

Binary Tree

For each successive element, descend the tree, putting lower numbers to the left, larger numbers to the right.

13	9	16	22	15	1	5	4

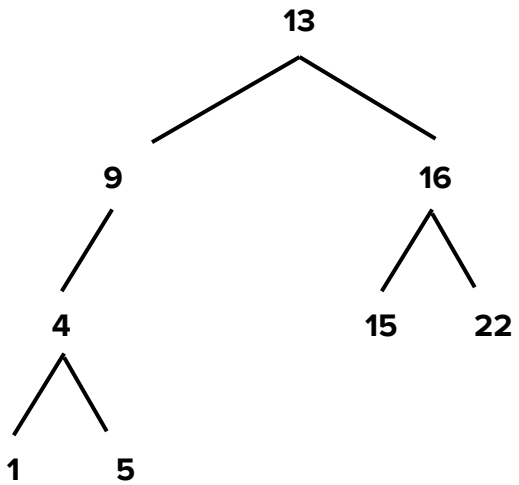


 © 2024 General Assembly

Binary Tree

For each successive element, descend the tree, putting lower numbers to the left, larger numbers to the right.

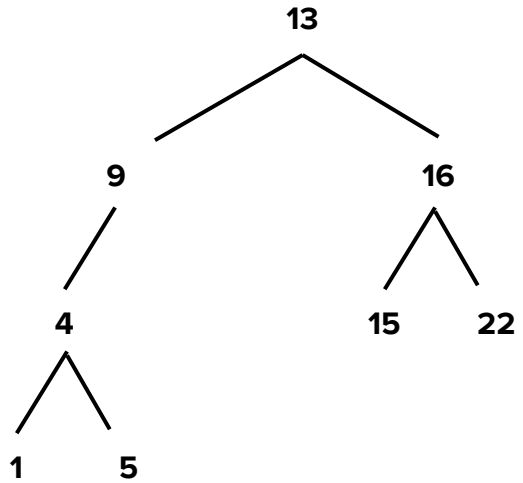
13	9	16	22	15	1	5	4



 © 2024 General Assembly

Binary Searching

Is 5 in this array (now a tree)?

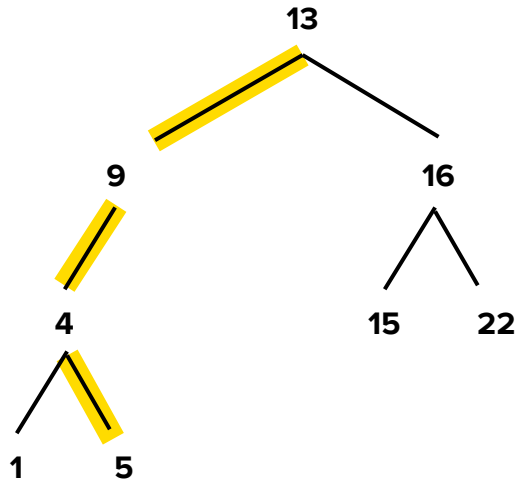


 © 2024 General Assembly

Binary Searching

Is 5 in this array (now a tree)?

Yes! We did 3 operations (left, left, right) and eventually found the 5!

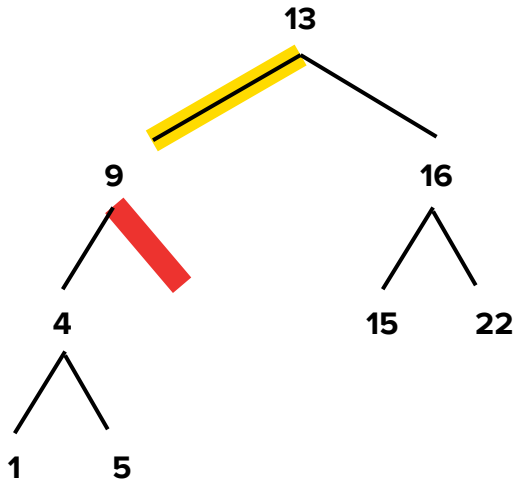


 © 2024 General Assembly

Binary Searching

Is 11 in this array (now a tree)?

No! We did 2 operations (left, right) and found the spot where an 11 would be if it existed, but it doesn't!



Creating Data Structures

Yes - there's more work here! We'll need to use OOP in Python to create a data structure and that takes time and effort too. We've basically broken our problem into two steps:

1. Create data structure
2. Run algorithm

These problems are meant to be solved at scale - we're spending time now (coding) to save time later (code runtime).

 © 2024 General Assembly

Let’s do it!

